Image classification

Image classification
1. Model introduction
2. Start
2.1. Enter docker
2.2. Image classification: image
Effect preview
2.3, Image classification: Video
Effect preview
2.4, Image classification: real-time detection
Effect preview
References

Use Python to demonstrate the effect of **Ultralytics**: Image classification on images, videos, and real-time detection.

1. Model introduction

Image classification is the simplest of the three tasks and involves classifying the entire image into one of a set of predefined categories.

The output of the image classifier is a single class label and a confidence score. Image classification is very useful when you only need to know which class the image belongs to, without knowing the location or exact shape of the object in that class.

2. Start

2.1. Enter docker

Run YOLOv11 docker script

```
sh ~/yolov11_dcoker.sh
```

2.2. Image classification: image

Use yolo11n-cls.pt to predict the images under the ultralytics project (not the images that come with ultralytics).

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

Run the code:

```
python3 04.classification_image.py
```

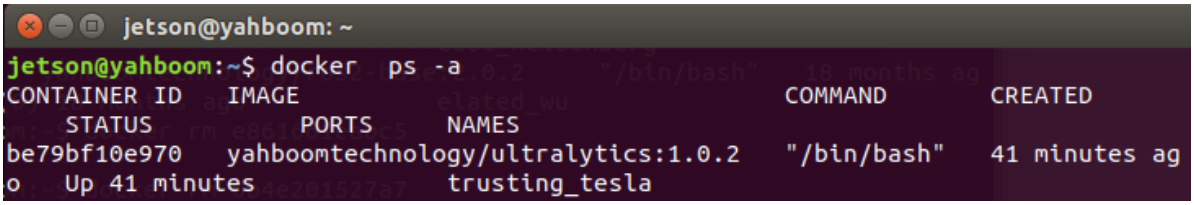
Effect preview

Yolo recognition output image location: /ultralytics/ultralytics/output/

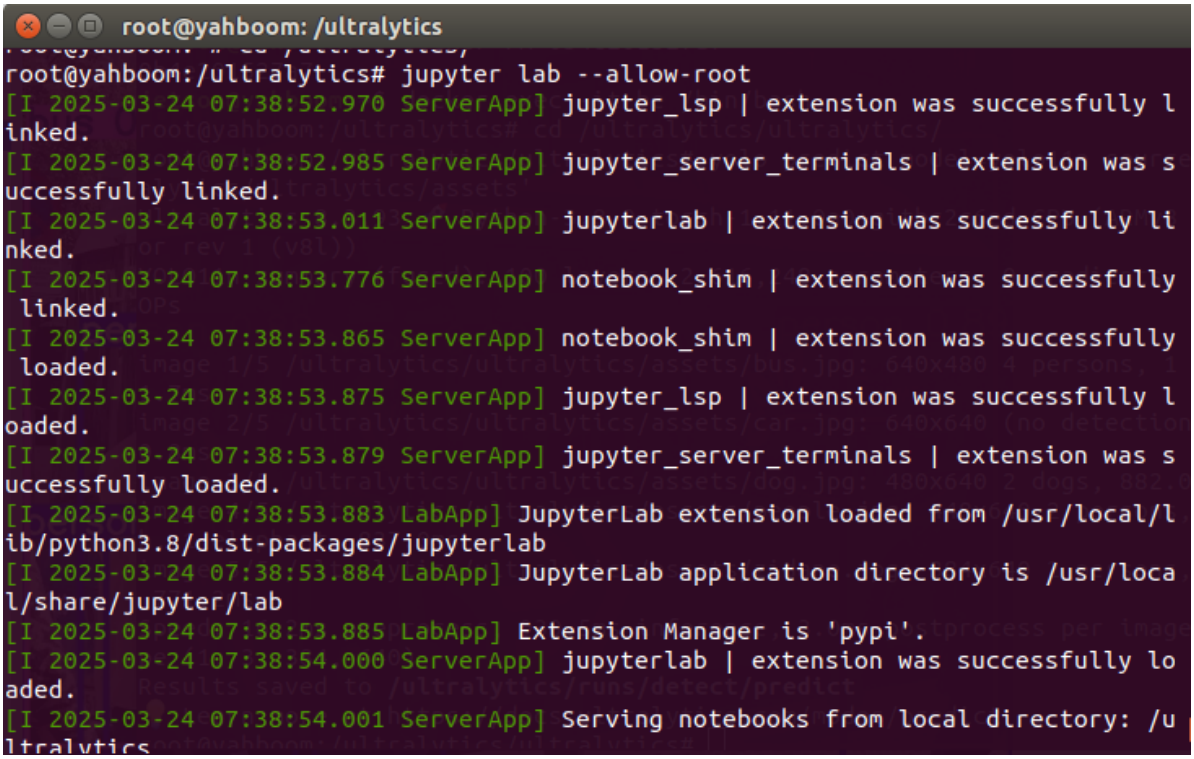
1. View using jupyter lab

Open another terminal to enter the docker container and use jupyter lab to view the image

```
docker ps -a
```

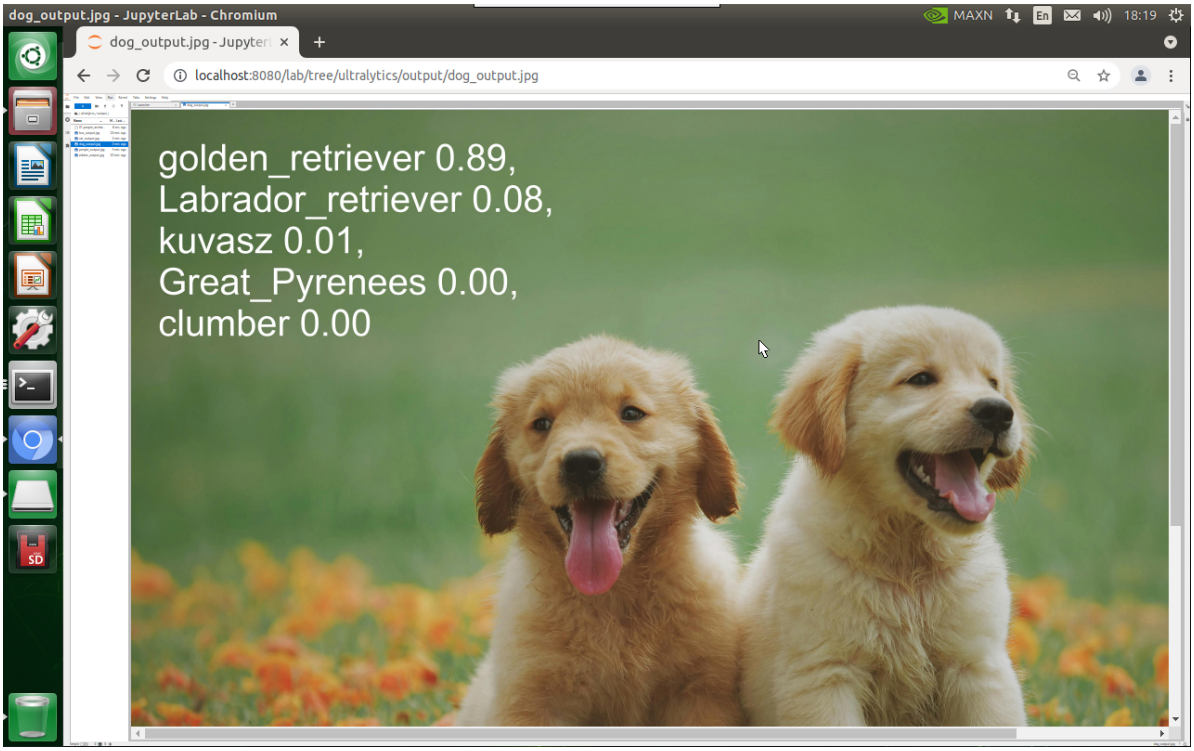



```
docker exec -it be79bf10e970 /bin/bash #Container ID needs to be modified
according to the actual one you find
cd /ultralytics
jupyter lab --allow-root
```



Access directly through http://localhost:8080/ in the system browser:

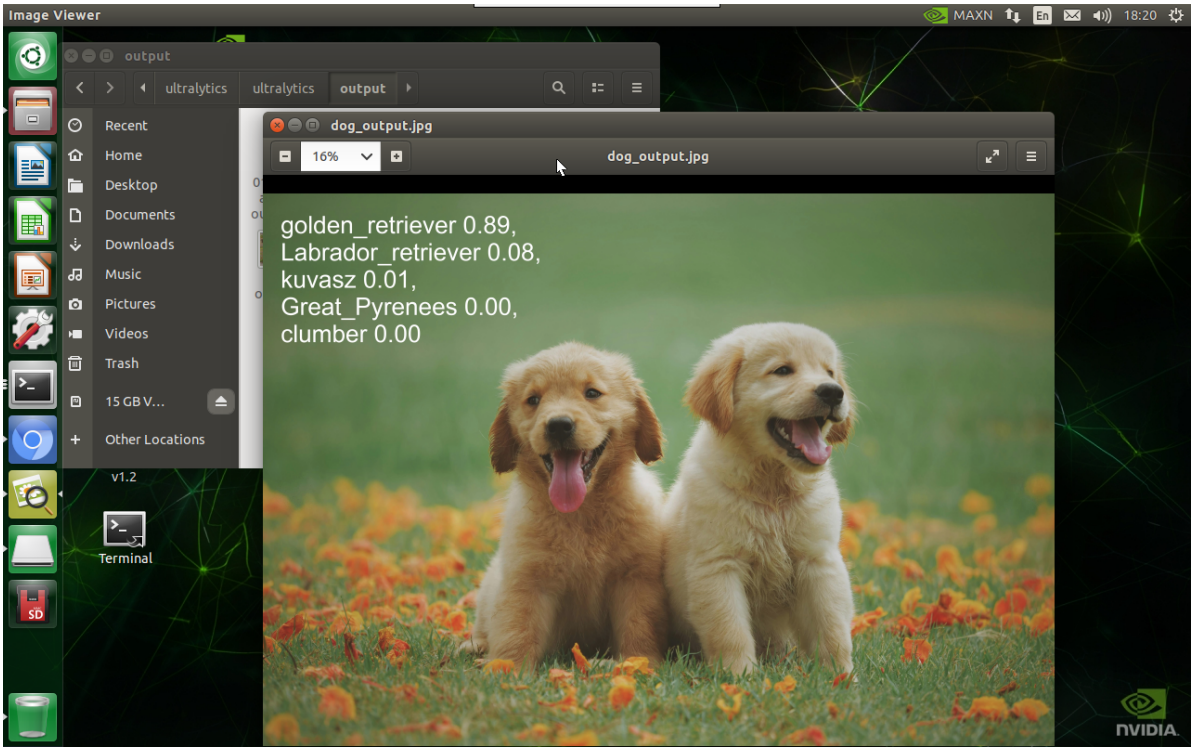
```
http://localhost:8080/
```



2. Copy the file to the host machine for viewing

Enter the following command in the host terminal

```
docker cp be79bf10e970:/ultralytics/ultralytics/output/
/home/jetson/ultralytics/ultralytics/ #Container ID needs to be modified
according to the actual one you find
```



Sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/ultralytics/ultralytics/yolo11n-cls.pt")
```

```
# Run batched inference on a list of images
results = model("/ultralytics/ultralytics/assets/dog.jpg")  # return a list of
Results objects

# Process results list
for result in results:
    # boxes = result.boxes  # Boxes object for bounding box outputs
    # masks = result.masks  # Masks object for segmentation masks outputs
    # keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Probs object for classification outputs
    # obb = result.obb  # Oriented boxes object for OBB outputs
    result.show()  # display to screen
    result.save(filename="/ultralytics/ultralytics/output/dog_output.jpg")  #
save to disk
```

2.3, Image classification: Video

Use yolo11n-cls.pt to predict videos under the ultralytics project (not the videos that come with ultralytics).

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

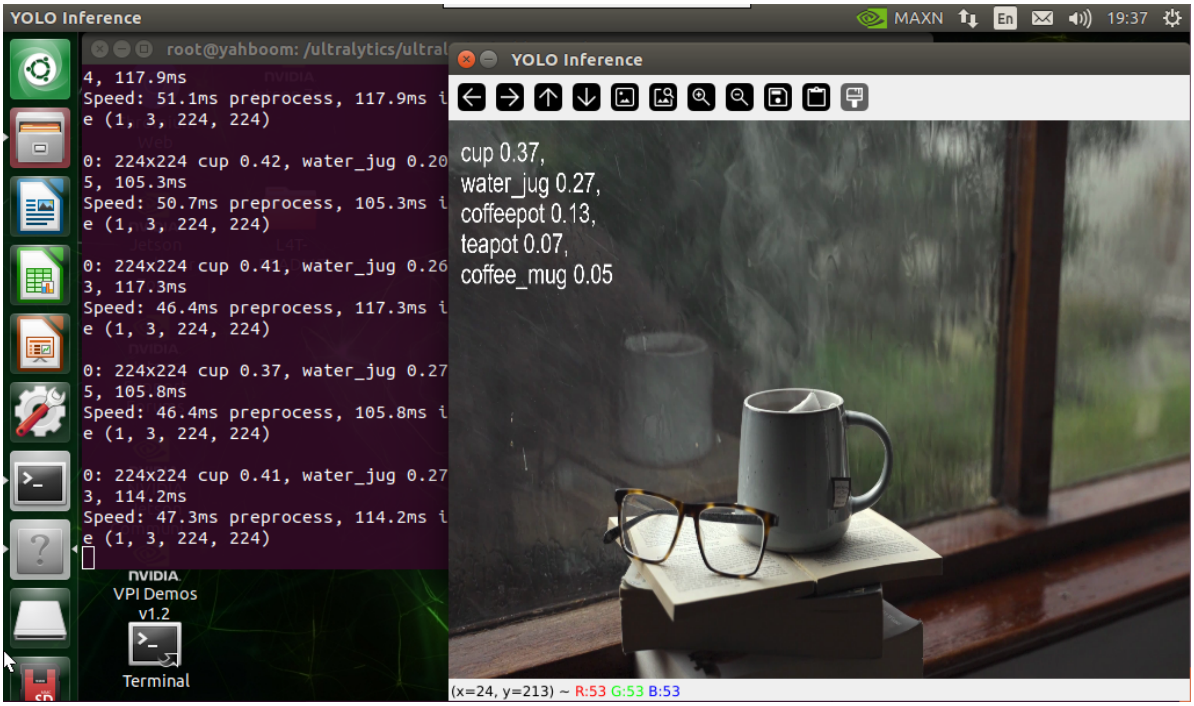
Run the code:

```
python3 04.classification_video.py
```

Effect preview

Video location of yolo recognition output: /ultralytics/ultralytics/output/

The output video will be displayed in real time during the code running process. If you want to view the video later, you can refer to the above **[2. Copy the file to the host machine for viewing]** tutorial operation.



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/ultralytics/yolo11n-cls.pt")

# Open the video file
video_path = "/ultralytics/ultralytics/videos/cup.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/ultralytics/ultralytics/output/04.cup_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

2.4, Image classification: real-time detection

Use yolo11n-cls.pt to predict the USB camera screen.

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

Run the code: Click the preview screen, press the q key to terminate the program!

```
python3 04.classification_camera_usb.py
```

Effect preview

Yolo recognizes the output video location: /ultralytics/ultralytics/output/

The camera screen will be displayed in real time during the code running. If you want to view the video later, you can refer to the above **[2. Copy the file to the host machine for viewing]** tutorial operation.



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/ultralytics/yolo11n-cls.pt")

# Open the cammera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))
```

```
# Define the codec and create a VideoWriter object to output the processed video
output_path = "/ultralytics/ultralytics/output/04.classification_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

References

https://docs.ultralytics.com/tasks/classify/